c0a251904e 左上のUIデザイン変更 f022b89 · 8 minutes ago

308 lines (241 loc) · 10.4 KB

Code Blame Raw

```python
import pygame
import sys
import os
import random

# カレントディレクトリの固定
os.chdir(os.path.dirname(os.path.abspath(__file__)))

WIDTH, HEIGHT = 600, 800
FPS = 60
GRAVITY = 0.6

BLACK = (20, 20, 25)
BLUE = (100, 150, 255)
GREEN = (120, 120, 120)
RED = (255, 100, 100)
WHITE = (255, 255, 255)
GOLD = (255, 215, 0)  # ゴールブロック用の金色

DARK_PANEL = (35, 35, 45)
PANEL_BORDER = (90, 90, 110)
LIGHT_BLUE = (120, 200, 255)                #左上UI
ORANGE = (255, 170, 80)


# ========================================
# 1. データ構造（クラス）の定義
# ========================================

class Player:
    def __init__(self):
        self._image = pygame.Surface((30, 40))
        self._image.fill(BLUE)
        self._rect = self._image.get_rect(center=(WIDTH //

        self._vel_x = 0
        self._vel_y = 0
        self._on_ground = False

        self._is_charging = False
        self._charge_power = 0
        self._max_charge = 20.0
```

All Symbols

random

4 References Search

In this file

```
4   import random
216 w = random.randint(60, 130)
217 x = random.randint(0, WIDTH - w)
219 gap_y = random.randint(80, 130)
```

Search for this symbol

```
43              self._direction = 1
44
45              self._is_clear = False  # ゴールしたかどうかのフラグ
46
47          # --- ゲッター ---
48          def get_image(self): return self._image
49          def get_rect(self): return self._rect
50          def get_vel_x(self): return self._vel_x
51          def get_vel_y(self): return self._vel_y
52          def get_on_ground(self): return self._on_ground
53          def get_is_charging(self): return self._is_charging
54          def get_charge_power(self): return self._charge_power
55          def get_max_charge(self): return self._max_charge
56          def get_direction(self): return self._direction
57          def get_is_clear(self): return self._is_clear
58
59          # --- セッター ---
60          def set_vel_x(self, value): self._vel_x = value
61          def set_vel_y(self, value): self._vel_y = value
62          def set_on_ground(self, value): self._on_ground = value
63          def set_is_charging(self, value): self._is_charging = v
64          def set_charge_power(self, value): self._charge_power =
65          def set_direction(self, value): self._direction = value
66          def set_is_clear(self, value): self._is_clear = value
67
68
69      class Platform:
70          # 足場ごとに色を変えられるように引数追加
71          def __init__(self, x, y, w, h, color):
72              self._image = pygame.Surface((w, h))
73              self._image.fill(color)
74              self._rect = self._image.get_rect(topleft=(x, y))
75
76          def get_image(self): return self._image
77          def get_rect(self): return self._rect
78
79
80      # ========================================
81      # 2. クラスを操作する関数
82      # ========================================
83
84      def update_player(player, keys, platforms, goal_block):
85          """プレイヤーの物理挙動と状態を更新する関数"""
86
87          # ゴール済みの場合は入力を受け付けない（重力で落ちるだけ）
88          if not player.get_is_clear():
89              # 1. 地面にいる時の処理
90              if player.get_on_ground():
91                  player.set_vel_x(0)
92
93                  if keys[pygame.K_LEFT] or keys[pygame.K_a]:
94                      player.set_direction(-1)
95                  if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
96                      player.set_direction(1)
97
98                  if keys[pygame.K_SPACE]:
99                      player.set_is_charging(True)
100                     current_power = player.get_charge_power()
101                     if current_power < player.get_max_charge():
```

```
102                     player.set_charge_power(current_power +
103                 else:
104                     if player.get_is_charging():
105                         power = player.get_charge_power()
106                         direction = player.get_direction()
107 
108                         player.set_vel_y(-power * 0.9 - 5)
109                         player.set_vel_x(direction * (power * 0
110 
111                         player.set_is_charging(False)
112                         player.set_charge_power(0)
113                         player.set_on_ground(False)
114 
115         # 2. 重力の適用
116         if not player.get_on_ground():
117             player.set_vel_y(player.get_vel_y() + GRAVITY)
118 
119         rect = player.get_rect()
120 
121         # 座標の更新（X軸）と壁の跳ね返り
122         rect.x += player.get_vel_x()
123         if rect.left < 0:
124             rect.left = 0
125             player.set_vel_x(player.get_vel_x() * -0.5)
126             player.set_direction(1)
127         if rect.right > WIDTH:
128             rect.right = WIDTH
129             player.set_vel_x(player.get_vel_x() * -0.5)
130             player.set_direction(-1)
131 
132         # 座標の更新（Y軸）
133         rect.y += int(player.get_vel_y())
134 
135         # --- 3. ゴール（天井）との当たり判定 ---
136         goal_rect = goal_block.get_rect()
137         if rect.colliderect(goal_rect):
138             # 上昇中に天井の底にぶつかったらゴール
139             if player.get_vel_y() < 0:
140                 rect.top = goal_rect.bottom
141                 player.set_vel_y(0)
142                 player.set_is_clear(True)
143 
144         # --- 4. 足場との当たり判定 ---
145         player.set_on_ground(False)
146         if player.get_vel_y() > 0:
147             for plat in platforms:
148                 plat_rect = plat.get_rect()
149                 if rect.colliderect(plat_rect):
150                     if rect.bottom <= plat_rect.top + player.ge
151                         rect.bottom = plat_rect.top
152                         player.set_vel_y(0)
153                         player.set_vel_x(0)
154                         player.set_on_ground(True)
155                         break
156 
157 def draw_ui(screen, font, player, current_floor, total_floo
158 
159         # UIのサイズ
160         panel_x = 10
```

```
        panel_y = 10
        panel_w = 300
        panel_h = 120

        # 背景のパネル
        panel_rect = pygame.Rect(panel_x, panel_y, panel_w, par
        pygame.draw.rect(screen, DARK_PANEL, panel_rect, border
        pygame.draw.rect(screen, PANEL_BORDER, panel_rect, 2, b
        # タイトル
        title_text = font.render("STATUS", True, GOLD)
        screen.blit(title_text, (panel_x + 15, panel_y + 10))

        # 階層
        floor_text = font.render(f"Floor {current_floor} / {tot
        screen.blit(floor_text, (panel_x + 15, panel_y + 42))

        # 高さ
        height_text = font.render(f"Height: {current_height //
        screen.blit(height_text, (panel_x + 15, panel_y + 70))

        # 最高到達点
        max_text = font.render(f"Best: {max_height // 10} m", T
        screen.blit(max_text, (panel_x + 180, panel_y + 70))




    # ========================================
    # 3. メインループ
    # ========================================

    def main():
        pygame.init()
        screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Jump King - 10 Floors to Go
        clock = pygame.time.Clock()
        font = pygame.font.SysFont(None, 36)
        large_font = pygame.font.SysFont(None, 72)

        player = Player()

        # --- 10層分のマップ生成 ---
        platforms = []
        platforms.append(Platform(0, HEIGHT - 40, WIDTH, 40, GR

        TOTAL_FLOORS = 10
        # ゴール（天井）のY座標：スタート床から 10層分(800px * 10)
        goal_y = (HEIGHT - 40) - (HEIGHT * TOTAL_FLOORS)

        # 金色の天井ブロックを生成
        goal_block = Platform(0, goal_y, WIDTH, 40, GOLD)

        # ゴールに到達するまで足場を生成し続ける
        platform_y = HEIGHT - 150
        while platform_y > goal_y + 100:
            w = random.randint(60, 130)
            x = random.randint(0, WIDTH - w)
            platforms.append(Platform(x, platform_y, w, 20, GRE
            gap_y = random.randint(80, 130)
```

```
            platform_y -= gap_y

        camera_y = 0
        max_height = 0

        running = True
        while running:
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    running = False

            keys = pygame.key.get_pressed()

            # 関数呼び出し（ゴールブロックも渡して判定させる）
            update_player(player, keys, platforms, goal_block)

            p_rect = player.get_rect()

            # --- カメラ処理 ---
            SCROLL_TOP_MARGIN = 200
            if p_rect.y - camera_y < SCROLL_TOP_MARGIN:
                camera_y = p_rect.y - SCROLL_TOP_MARGIN

            SCROLL_BOTTOM_MARGIN = 150
            camera_easing_down = 0.08
            if p_rect.bottom - camera_y > HEIGHT - SCROLL_BOTTO
                target_camera_y = p_rect.bottom - (HEIGHT - SCR
                camera_y += (target_camera_y - camera_y) * came

            if camera_y > 0:
                camera_y = 0

            # --- スコア・階層計算 ---
            current_height = (HEIGHT - 40) - p_rect.bottom
            if current_height > max_height:
                max_height = current_height

            # 今いる階層（800px を 1階 とする）
            current_floor = min((current_height // HEIGHT) + 1,

            # --- 描画処理 ---
            screen.fill(BLACK)

            # ゴールの描画
            goal_rect = goal_block.get_rect()
            goal_draw_y = goal_rect.y - camera_y
            if -100 < goal_draw_y < HEIGHT + 100:
                screen.blit(goal_block.get_image(), (goal_rect.

            # 足場の描画
            for plat in platforms:
                plat_rect = plat.get_rect()
                draw_y = plat_rect.y - camera_y
                if -50 < draw_y < HEIGHT + 50:
                    screen.blit(plat.get_image(), (plat_rect.x,

            # プレイヤーの描画
            player_draw_y = p_rect.y - camera_y
            screen.blit(player.get_image(), (p_rect.x, player_d
```

```
279
280             # チャージゲージと方向指示器
281             if player.get_is_charging() and not player.get_is_c
282                 gauge_width = (player.get_charge_power() / play
283                 pygame.draw.rect(screen, RED, (p_rect.centerx -
284
285             arrow_x = p_rect.right + 5 if player.get_direction(
286             pygame.draw.rect(screen, WHITE, (arrow_x, player_dr
287
288
289             # 情報表示 (UI)
290             draw_ui(screen, font, player, current_floor, TOTAL_
291             # ゴールした時の演出
292             if player.get_is_clear():
293                 clear_text = large_font.render("CLEAR!!", True,
294                 text_rect = clear_text.get_rect(center=(WIDTH /
295                 screen.blit(clear_text, text_rect)
296
297                 sub_text = font.render("CONGRATULATIONS!", True
298                 sub_rect = sub_text.get_rect(center=(WIDTH // 2
299                 screen.blit(sub_text, sub_rect)
300
301             pygame.display.flip()
302             clock.tick(FPS)
303
304         pygame.quit()
305         sys.exit()
306
307     if __name__ == "__main__":
308         main()
```